Week4 : Lists, Objectives

Upon completion of this unit, students will be able to understand and explain the following sub topics :

- Lists
- A List Implementation that Uses an Array
- A List Implementation That Links Data

Lists

- A way to organize data
- Examples
  - To-do list
  - Gift lists
  - Grocery Lists
- Items in list have position
  - May or may not be important
- Items may be added anywhere

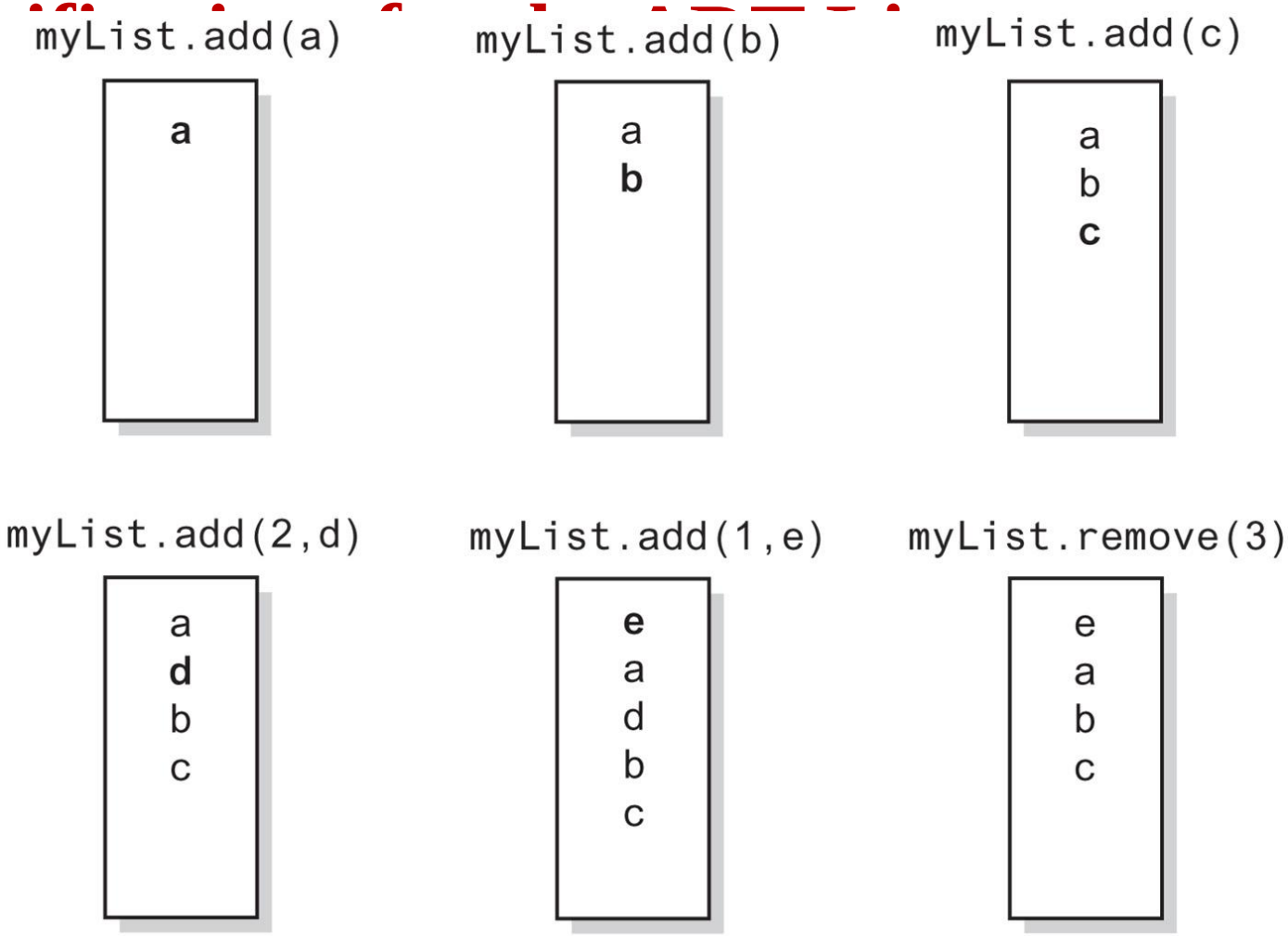
FIGURE 10-1
A to-do list

Specifications for the ADT List

- `add(newEntry)`
- `add(newPosition, newEntry)`
- `remove(givenPosition)`
- `clear()`
- `replace(givenPosition, newEntry)`
- `getEntry(givenPosition)`
- `toArray()`
- `contains(anEntry)`
- `getLength()`
- `isEmpty()`

Spe

- FIGU
  list



npty

© 2019 Pearson Education, Inc.

Specifications for the ADT List

- Data
  - A collection of objects in a specific order and having the same data type
  - The number of objects in the collection

add(newEntry)	**+add(newEntry: T): void**	Task: Adds newEntry to the end of the list. Input: newEntry is an object. Output: None.
add(newPosition, anEntry)	**+add(newPosition: integer, anEntry: T): void**	Task: Adds newEntry at position newPosition within the list. Position 1 indicates the first entry in the list. Input: newPosition is an integer, newEntry is an object. Output: Throws an exception if newPosition is invalid for this list before the operation.
remove(givenPosition)	**+remove(givenPosition: integer): T**	Task: Removes and returns the entry at position givenPosition. Input: givenPosition is an integer. Output: Either returns the removed entry or throws an exception if givenPosition is invalid for this list. Note that any value of givenPosition is invalid if the list is empty before the operation.
clear()	**+clear(): void**	Task: Removes all entries from the list. Input: None. Output: None.
replace(givenPosition, anEntry)	**+replace(givenPosition: integer, anEntry: T): T**	Task: Replaces the entry at position givenPosition with newEntry. Input: givenPosition is an integer, newEntry is an object. Output: Either returns the replaced entry or throws an exception if givenPosition is invalid for this list. Note that any value of givenPosition is invalid if the list is empty before the operation.

Specifications for the ADT List

- Data
  - A collection of objects in a specific order and having the same data type
  - The number of objects in the collection

getEntry(givenPosition)	**+getEntry(givenPosition: integer): T**	Task: Retrieves the entry at position givenPosition. Input: givenPosition is an integer. Output: Either returns the entry at position givenPosition or throws an exception if givenPosition is invalid for this list. Note that any value of givenPosition is invalid if the list is empty before the operation.
toArray()	**+toArray: T[ ]**	Task: Retrieves all entries that are in the list in the order in which they occur. Input: None. Output: Returns a new array of the entries currently in the list.
contains(anEntry)	**+contains(anEntry: T): boolean**	Task: Sees whether the list contains anEntry. Input: anEntry is an object. Output: Returns true if anEntry is in the list, or false if not.
getLength()	**+getLength(): integer**	Task: Gets the number of entries currently in the list. Input: None. Output: Returns the number of entries currently in the list.
isEmpty()	**+isEmpty(): boolean**	Task: Sees whether the list is empty. Input: None. Output: Returns true if the list is empty, or false if not.

LISTING 10-1 The interface `ListInterface` (Part 1)

```
** An interface ADT list. Entries in a list have positions that begin with 1.  */
public interface ListInterface<T> {
 /** Adds a new entry to the end of this list.
     Entries currently in the list are unaffected.
     The list's size is increased by 1.
     @param newEntry  The object to be added as a new entry. */
  public void add(T newEntry);

  /** Adds a new entry at a specified position within this list.
     Entries originally at and above the specified position
     are at the next higher position within the list.
     The list's size is increased by 1.
     @param newPosition  An integer that specifies the desired
                         position of the new entry.
     @param newEntry     The object to be added as a new entry.
     @throws  IndexOutOfBoundsException if either
              newPosition < 1 or newPosition > getLength() + 1. */
  public void add(int newPosition, T newEntry);

  /** Removes the entry at a given position from this list.
     Entries originally at positions higher than the given
     position are at the next lower position within the list,
     and the list's size is decreased by 1.
     @param givenPosition  An integer that indicates the position of
                           the entry to be removed.
     @return  A reference to the removed entry.
     @throws  IndexOutOfBoundsException if either
              givenPosition < 1 or givenPosition > getLength(). */
  public T remove(int givenPosition);
```

LISTING 10-1 The interface `ListInterface` (Part 2)

```
/** Removes all entries from this list. */
public void clear();

/** Replaces the entry at a given position in this list.
   @param givenPosition  An integer that indicates the position of
                         the entry to be replaced.
   @param newEntry  The object that will replace the entry at the
                    position givenPosition.
   @return  The original entry that was replaced.
   @throws  IndexOutOfBoundsException if either
            givenPosition < 1 or givenPosition > getLength(). */
public T replace(int givenPosition, T newEntry);

/** Retrieves the entry at a given position in this list.
   @param givenPosition  An integer that indicates the position of
                         the desired entry.
   @return  A reference to the indicated entry.
   @throws  IndexOutOfBoundsException if either
            givenPosition < 1 or givenPosition > getLength(). */
public T getEntry(int givenPosition);

/** Retrieves all entries that are in this list in the order in which
   they occur in the list.
   @return  A newly allocated array of all the entries in the list.
            If the list is empty, the returned array is empty. */
public T[] toArray();

/** Sees whether this list contains a given entry.
   @param anEntry  The object that is the desired entry.
   @return  True if the list contains anEntry, or false if not. */
public boolean contains(T anEntry);
```

LISTING 10-1 The interface `ListInterface` (Part 3)

```
   /** Gets the length of this list.
       @return  The integer number of entries currently in the list. */
   public int getLength();

   /** Sees whether this list is empty.
       @return  True if the list is empty, or false if not. */
   public boolean isEmpty();
} // end ListInterface
```

Using

16
4
33
27

© 2019 Pearson Education, Inc.

- FIGURE 10-3 A list of numbers that identify runners in the order in which they finished

Using the ADT List

```
/**   A driver that uses a list to track the
   runners in a race as they cross the finish line. */
public class RoadRace {
      public static void main(String[] args)  {
      recordWinners();
      }  // end main

      public static void recordWinners() {
     ListInterface<String> runnerList = new AList<>();
  //  runnerList has only methods in ListInterface

     runnerList.add("16"); // Winner
     runnerList.add(" 4"); // Second place
     runnerList.add("33"); // Third place
     runnerList.add("27"); // Fourth place
     displayList(runnerList);
   } // end recordWinners

   public static void displayList(ListInterface<String> list) {
     int numberOfEntries = list.getLength();
     System.out.println("The list contains " + numberOfEntries +
                        " entries, as follows:");
     for (int position = 1; position <= numberOfEntries; position++)
        System.out.println(list.getEntry(position) +
                           " is entry " + position);
     System.out.println();
   } // end displayList
} // end RoadRace
```

Program Output

```
The list contains 4 entries,
as follows:
16 is entry 1
 4 is entry 2
33 is entry 3
27 is entry 4
```

Using the ADT List

- Example

```
// Make an alphabetical list of names as students enter a room
ListInterface<String> alphaList = new AList<>();

alphaList.add(1, "Amy");    // Amy
alphaList.add(2, "Elias");  // Amy Elias
alphaList.add(2, "Bob");    // Amy Bob Elias
alphaList.add(3, "Drew");   // Amy Bob Drew Elias
alphaList.add(1, "Aaron");  // Aaron Amy Bob Drew Elias
alphaList.add(4, "Carol");  // Aaron Amy Bob Carol Drew Elias
```

Using the ADT List

- A list of Name objects, rather than String

```
// Make a list of names as you think of them
ListInterface<Name> nameList = new AList<>();
Name amy = new Name("Amy", "Smith");
nameList.add(amy);
nameList.add(new Name("Tina", "Drexel");
nameList.add(new Name("Robert", "Jones");

Name secondName = nameList.getEntry(2);
```

Java Class Library: The Interface `List`

- Method headers from the interface List

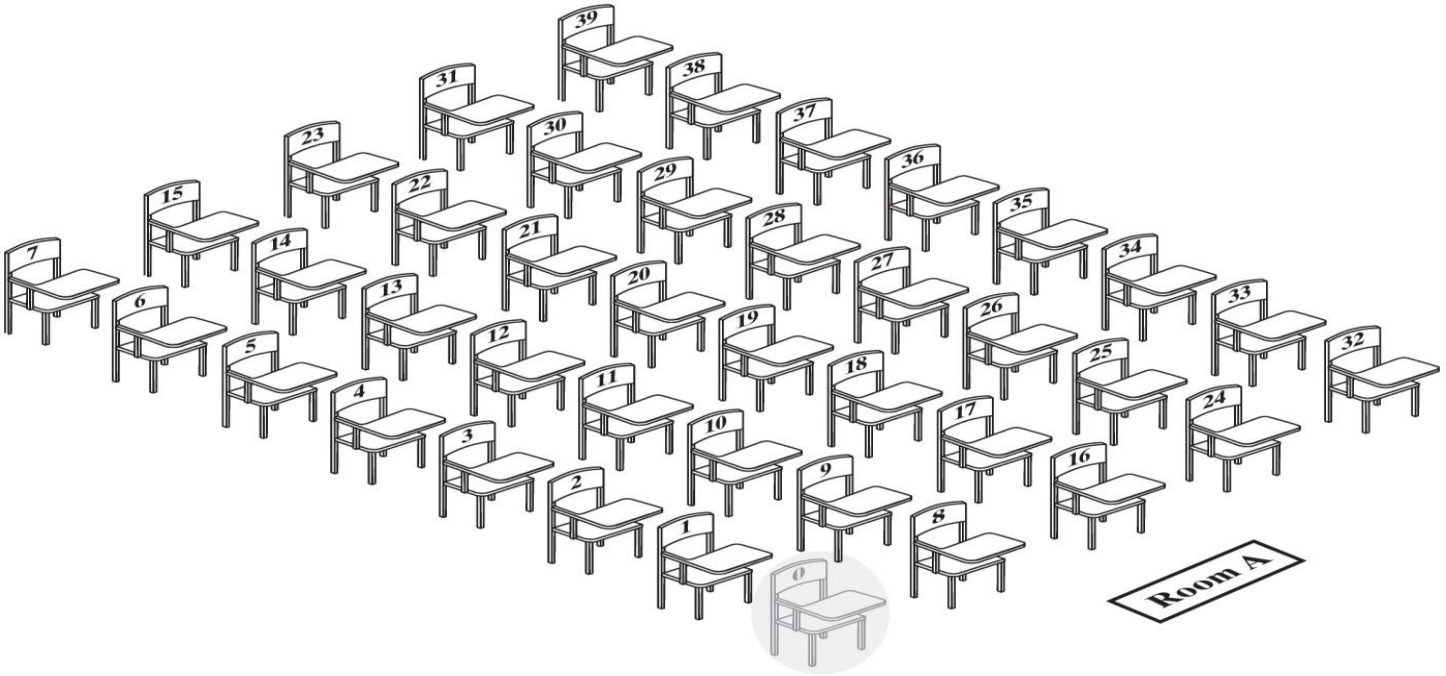
```
public void add(int index, T newEntry)
public T remove(int index)
public void clear()
public T set(int index, T anEntry) // Like replace
public  T get(int index) // Like getEntry
public boolean contains(Object anEntry)
public int size() // Like getLength
public boolean isEmpty()
```

Java Class Library: The Class `ArrayList`

- Available constructors
  - `public ArrayList()`
  - `public ArrayList(int initialCapacity)`
- Similar to `java.util.vector`
  - Can use either `ArrayList` or `Vector` as an implementation of the interface `List`.

Using an Array to Implement the ADT List

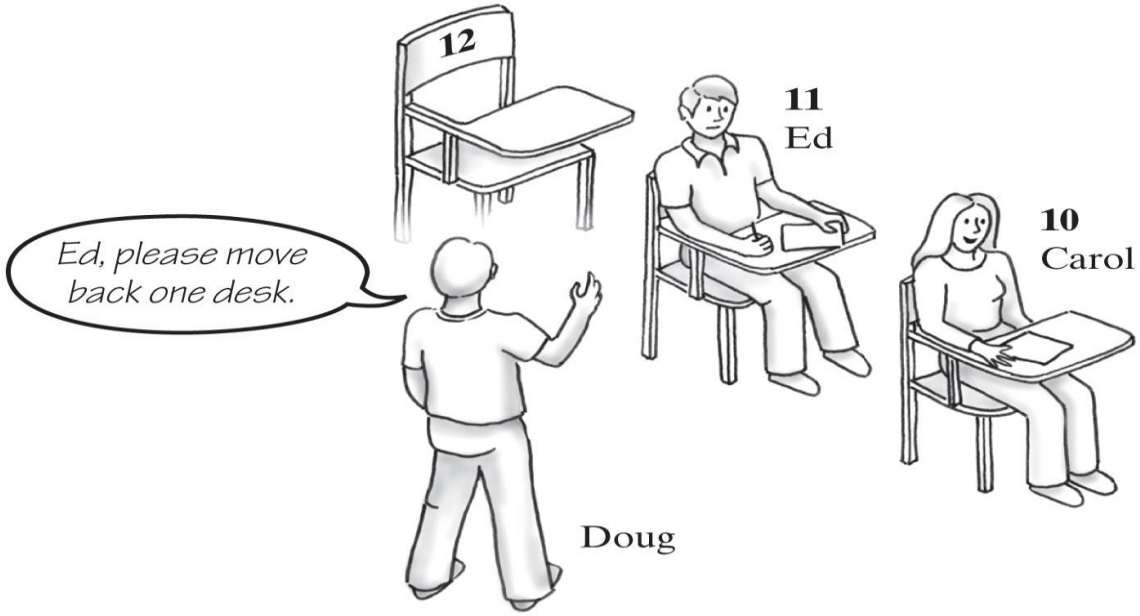
- A classroom that contains desks in fixed positions



© 2019 Pearson Education, Inc.

Using an Array to Implement the ADT List

- Seating a new student between two existing students: At least one other student must move



© 2019 Pearson Education, Inc.

Using an Array to Implement the ADT List

- UML notation for the class AList

```
AList
-list: T[]
-numberOfEntries: integer
-DEFAULT_CAPACITY: integer
-MAX_CAPACITY: integer
-integrityOK: boolean

+add(newEntry: T): void
+add(givenPosition: integer, newEntry: T): void
+remove(givenPosition: integer): T
+clear(): void
+replace(givenPosition: integer, newEntry: T): T
+getEntry(givenPosition: integer): T
+toArray(): T[]
+contains(anEntry: T): boolean
+getLength(): integer
+isEmpty(): boolean
```

An Array List Implementation (Part 1)

- LISTING 11-1 The class `AList`

```
/**
   A class that implements a list of objects by using an array.
   Entries in a list have positions that begin with 1.
   Duplicate entries are allowed. */
public class AList<T> implements ListInterface<T>
{
      private T[] list;   // Array of list entries; ignore list[0]
      private int numberOfEntries;
   private boolean integrityOK;
      private static final int DEFAULT_CAPACITY = 25;
      private static final int MAX_CAPACITY = 10000;

      public AList()
      {
      this(DEFAULT_CAPACITY);
      } // end default constructor
```

An Array List Implementation (Part 2)

- The class `AList`

```
 public AList(int initialCapacity)
 {
integrityOK = false;

// Is initialCapacity too small?
if (initialCapacity < DEFAULT_CAPACITY)
   initialCapacity = DEFAULT_CAPACITY;
else // Is initialCapacity too big?
   checkCapacity(initialCapacity);

// The cast is safe because the new array contains null entries
@SuppressWarnings("unchecked")
T[] tempList = (T[])new Object[initialCapacity + 1];
list = tempList;
numberOfEntries = 0;
integrityOK = true;
 } // end constructor

 public void add(T newEntry)
 {
 checkIntegrity();
 list[numberOfEntries + 1] = newEntry;
 numberOfEntries++;
 ensureCapacity();
//  add(numberOfEntries + 1, newEntry);  // ALTERNATE CODE
 } // end add
```

An Array List Implementation (Part 3)

- LISTING 1-1 The class `AList`

```
public void add(int newPosition, T newEntry)
{  /* < Implementation deferred > */
} // end add

public T remove(int givenPosition)
{ /* < Implementation deferred > */
} // end remove

public void clear()
{ /* < Implementation deferred > */
} // end clear

public T replace(int givenPosition, T newEntry)
{ /* < Implementation deferred > */
} // end replace

public T getEntry(int givenPosition)
{ /* < Implementation deferred > */
} // end getEntry
```

An Array List Implementation (Part 4)

- LISTING The class `AList`

```
public T[] toArray()
{
    checkIntegrity();

   // The cast is safe because the new array contains null entries
   @SuppressWarnings("unchecked")
   T[] result = (T[])new Object[numberOfEntries]; // Unchecked cast
   for (int index = 0; index < numberOfEntries; index++)
   {
      result[index] = list[index + 1];
   } // end for

   return result;
} // end toArray

    public boolean contains(T anEntry)
    { /* < Implementation deferred > */
    } // end contains

    public int getLength()
    {
    return numberOfEntries;
    } // end getLength
```

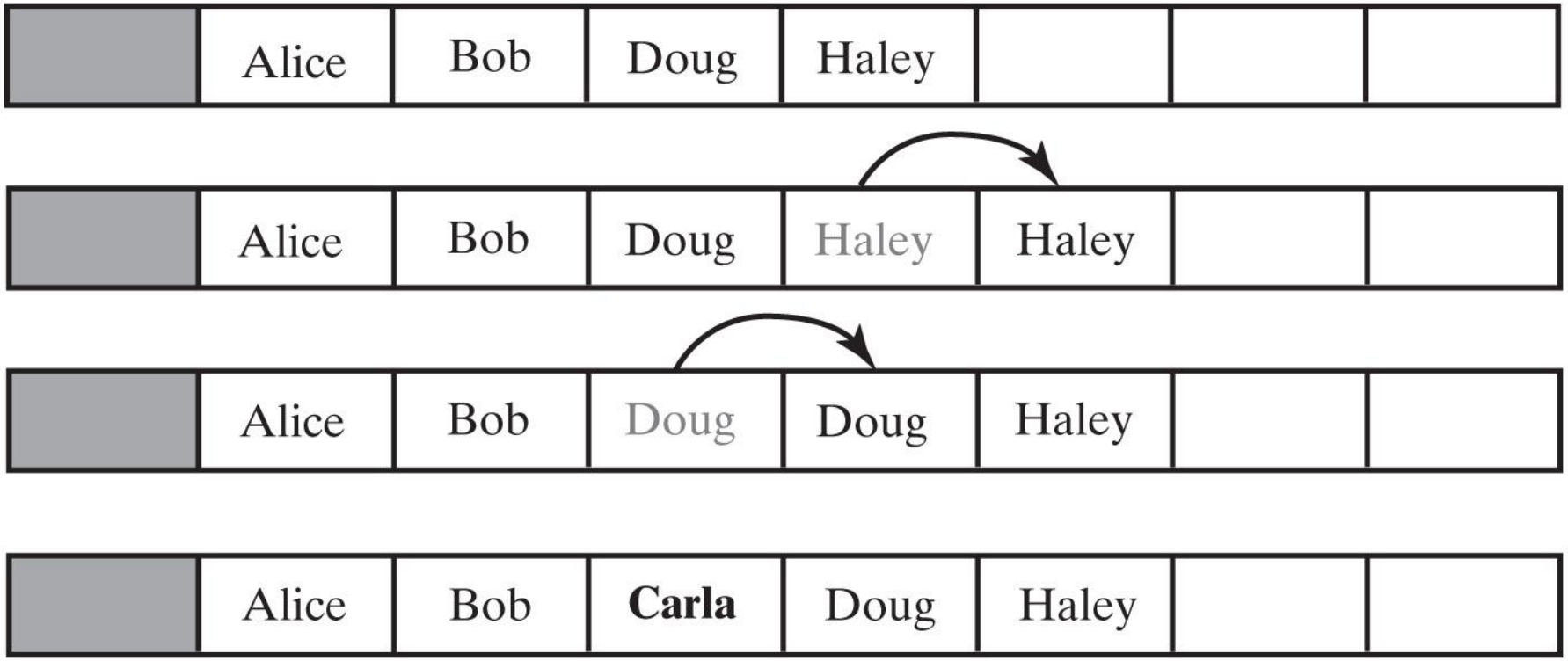
An Array List Implementation (Part 5)

- LISTING The class `AList`

```
    public int getLength()
    {
    return numberOfEntries;
    } // end getLength

    public boolean isEmpty()
    {
    return numberOfEntries == 0; // Or getLength() == 0
    } // end isEmpty

// Doubles the capacity of the array list if it is full.
// Precondition: checkIntegrity has been called.
private void ensureCapacity()
{
   int capacity = list.length - 1;
   if (numberOfEntries >= capacity)
   {
      int newCapacity = 2 * capacity;
      checkCapacity(newCapacity); // Is capacity too big?
      list = Arrays.copyOf(list, newCapacity + 1);
   } // end if
} // end ensureCapacity
```

Using an Array to Implement the ADT List

- Making room to insert Carla as the third entry in an array



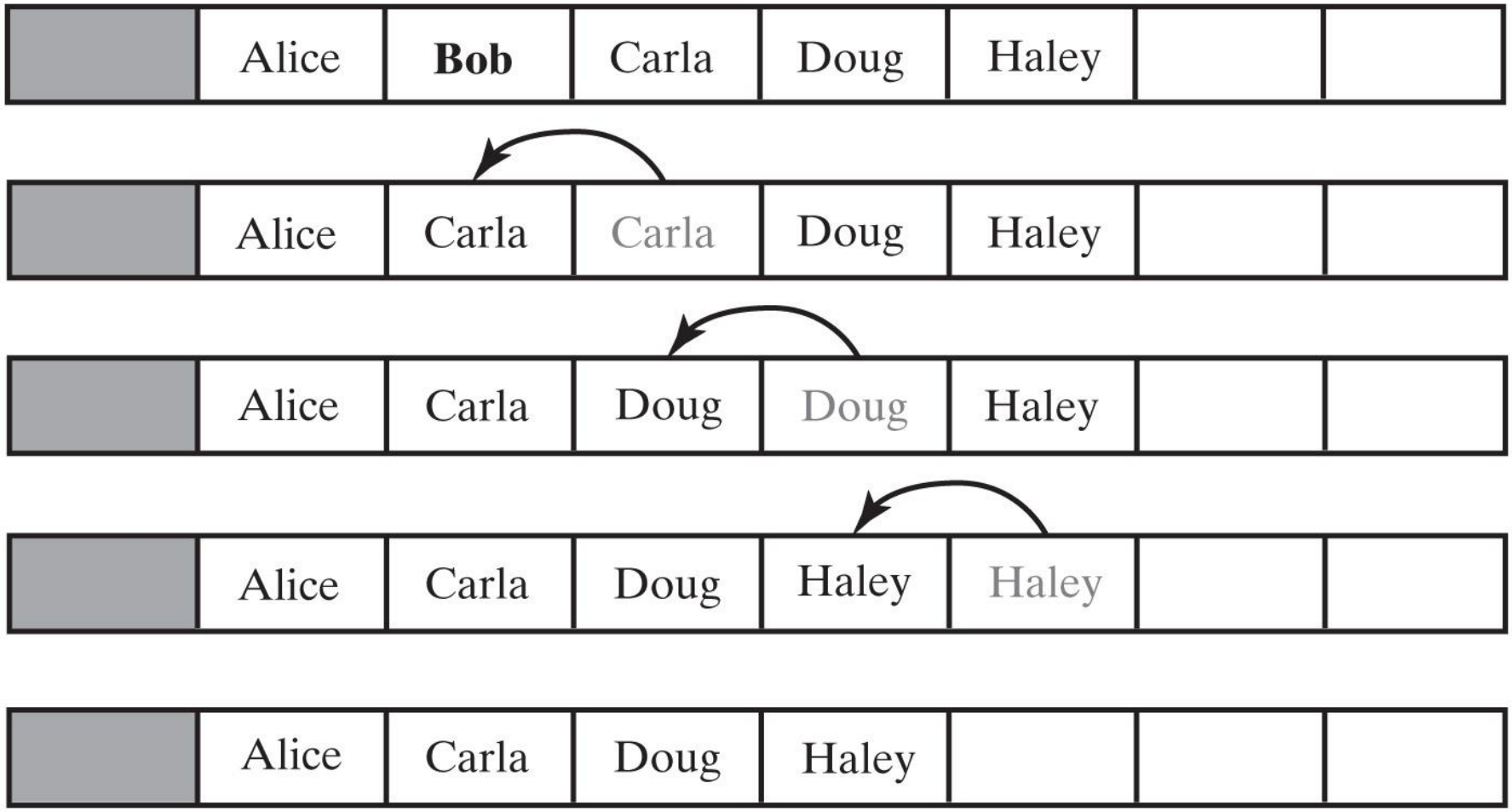
© 2019 Pearson Education, Inc.

Using an Array to Implement the ADT List

```
// Precondition: The array list has room for another entry.
public void add(int newPosition, T newEntry)
{
   checkIntegrity();
   // Assertion: The array list has room for another entry.
   if ((newPosition >= 1) && (newPosition <= numberOfEntries + 1))
   {
      if (newPosition <= numberOfEntries)
         makeRoom(newPosition);
      list[newPosition] = newEntry;
      numberOfEntries++;
      ensureCapacity(); // Ensure enough room for next add
   }
   else
      throw new IndexOutOfBoundsException(
               "Given position of add's new entry is out of bounds.");
} // end add
```

- Implementing `add` at a specific position

Using an Array to Implement the ADT List



© 2019 Pearson Education, Inc.

Using an Array to Implement the ADT List

```
public T remove(int givenPosition)
{
   checkIntegrity();
   if ((givenPosition >= 1) && (givenPosition <= numberOfEntries))
   {
      // Assertion: The list is not empty
      T result = list[givenPosition]; // Get entry to be removed
      // Move subsequent entries towards entry to be removed,
      // unless it is last in list
      if (givenPosition < numberOfEntries)
         removeGap(givenPosition);
      list[numberOfEntries] = null;
      numberOfEntries--;
      return result; // Return reference to removed entry
   }
   else
      throw new IndexOutOfBoundsException(
               "Illegal position given to remove operation.");
} // end remove
```

- Implementation uses a private method `removeGap` to handle the details of moving data within the array.

Using an Array to Implement the ADT List

- Method `removeGap` shifts list entries within the array

```
// Shifts entries that are beyond the entry to be removed to the
// next lower position.
// Precondition: 1 <= givenPosition < numberOfEntries;
//               numberOfEntries is list's length before removal;
//               checkIntegrity has been called.
private void removeGap(int givenPosition)
{
   int removedIndex = givenPosition;
   for (int index = removedIndex; index < numberOfEntries; index++)
      list[index] = list[index + 1];
} // end removeGap
```

Using an Array to Implement the ADT List

```
public boolean contains(T anEntry)
{
   checkIntegrity();
   boolean found = false;
   int index = 1;
   while (!found && (index <= numberOfEntries))
   {
      if (anEntry.equals(list[index]))
         found = true;
      index++;
   } // end while
   return found;
} // end contains
```

- Method `contains` uses a local boolean variable to terminate the loop when we find the desired entry.

Using an Array to Implement the ADT List

- Operation that adds a new entry to the end of a list. Efficiency O(1) if new if array is not resized.

```
public void add(T newEntry)
{
   checkIntegrity();
   list[numberOfEntries + 1] = newEntry;
   numberOfEntries++;
   ensureCapacity();
} // end add
```

Using an Array to Implement the ADT List

- Add a new entry to a list at a client-specified position.

```
public void add(int givenPosition, T newEntry)
{
   checkIntegrity();
   if ((givenPosition >= 1) && (givenPosition <= numberOfEntries + 1))
   {
      if (givenPosition <= numberOfEntries)
         makeRoom(givenPosition);
      list[givenPosition] = newEntry;
      numberOfEntries++;
      ensureCapacity(); // Ensure enough room for next add
   }
   else
      throw new IndexOutOfBoundsException(
               "Given position of add's new entry is out of bounds.");
} // end add
```

Using an Array to Implement the ADT List

- Method add uses method makeRoom.

```
private void makeRoom(int givenPosition)
{
   int newIndex = givenPosition;
   int lastIndex = numberOfEntries;
   for (int index = lastIndex; index >= newIndex; index--)
      list[index + 1] = list[index];
}  // end makeRoom
```

Advantages of Linked Implementation

- Uses memory only as needed
- When entry removed, unneeded memory returned to system
- Avoids moving data when adding or removing entries

Adding a Node at Various Positions

- Possible cases:
  - Chain is empty
  - Adding node at chain's beginning
  - Adding node between adjacent nodes
  - Adding node to chain's end

- This pseudocode establishes a new node for the given data

Adding a Node

newNode *references a new instance of* Node

Place newEntry *in* newNode

firstNode = *address of* newNode

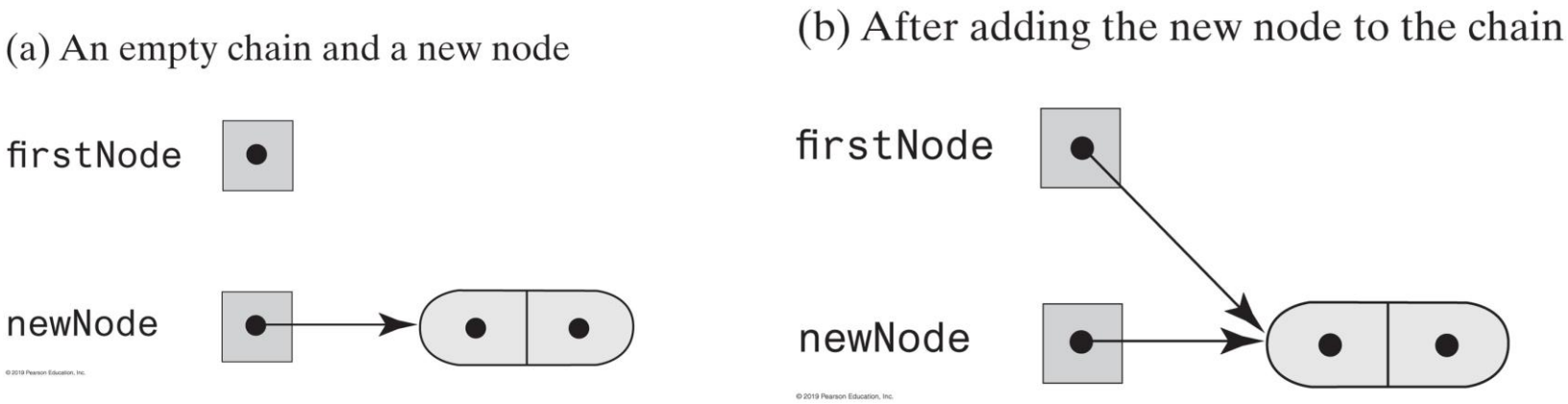


FIGURE 12-1 Adding a node to an empty chain

- This pseudocode describes the steps needed to add a node to the beginning of a chain.

Adding a Node

newNode *references a new instance of* Node

Place newEntry *in* newNode

Set newNode*'s link to* firstNode

Set firstNode *to* newNode

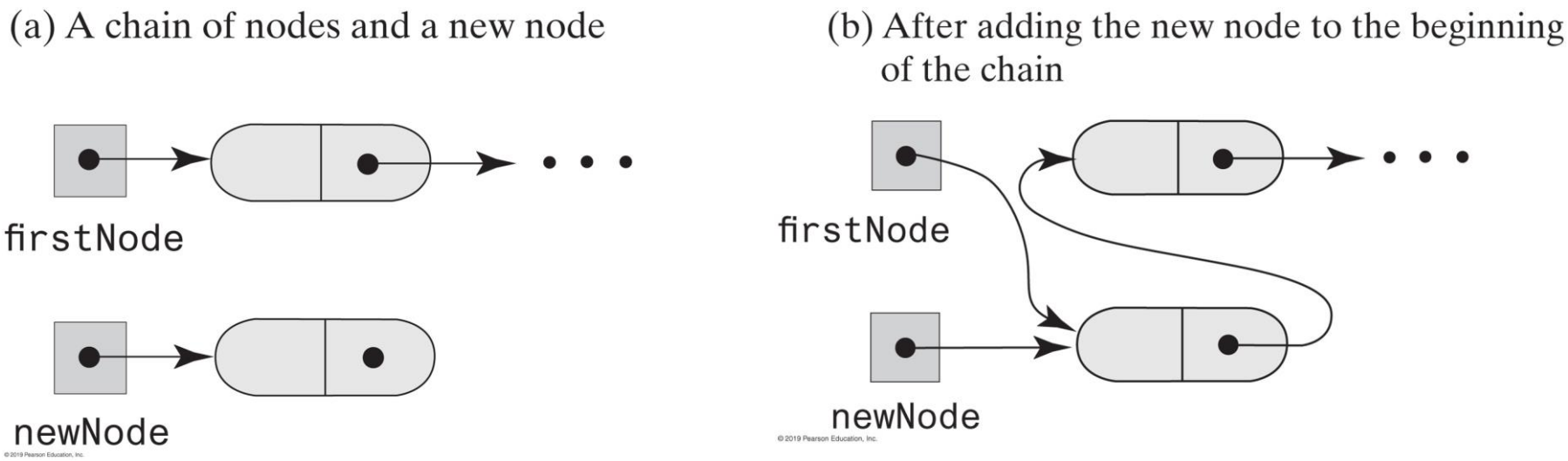


FIGURE 12-2 Adding a node to the beginning of a chain

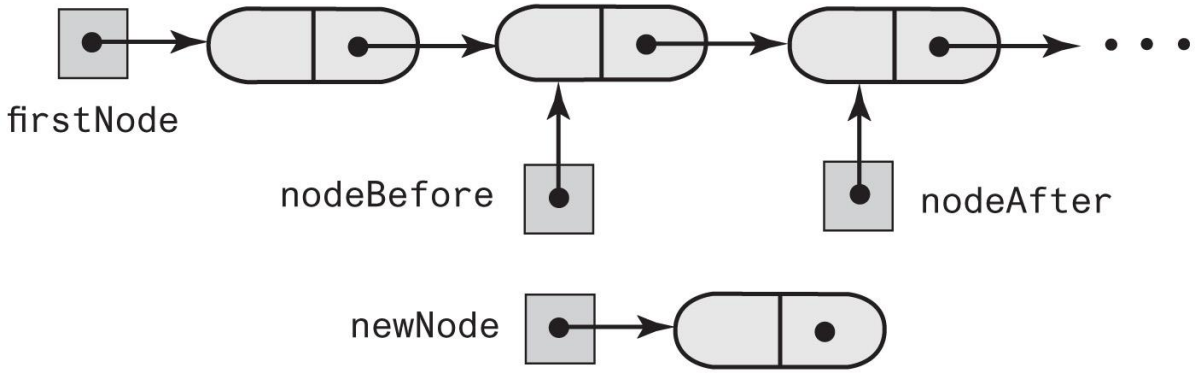
Adding a Node

- Pseudocode to add a node to a chain between two existing, consecutive nodes

newNode *references the new node*
Place newEntry *in* newNode
Let nodeBefore *reference the node that will be before the new node*
Set nodeAfter *to* nodeBefore*'s link*
Set newNode*'s link to* nodeAfter
Set nodeBefore*'s link to* newNode

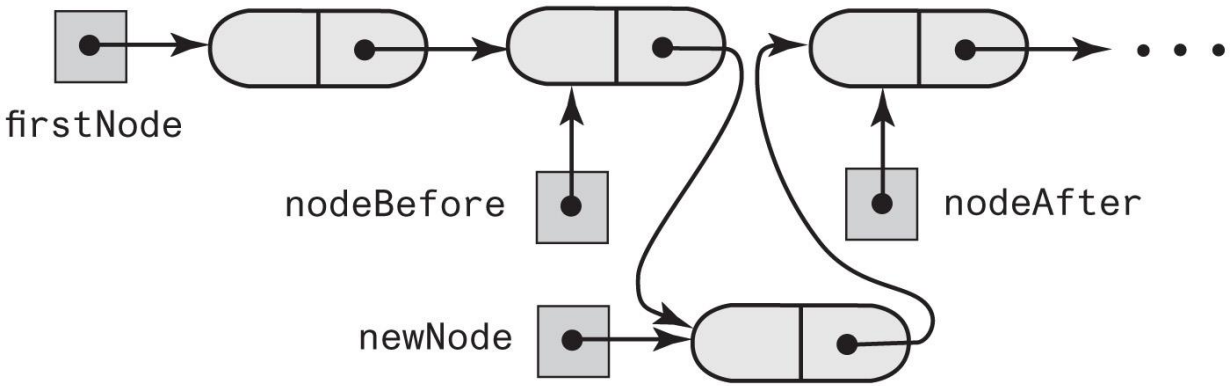
Add

(a) A chain of nodes and a new node



© 2019 Pearson Education, Inc.

(b) After adding the new node between adjacent nodes



© 2019 Pearson Education, Inc.

- Steps to add a node at the end of a chain.

Adding a Node

newNode *references a new instance of* Node

Place newEntry *in* newNode

Locate the last node in the chain

Place the address of newNode *in this last node*

Removing a Node

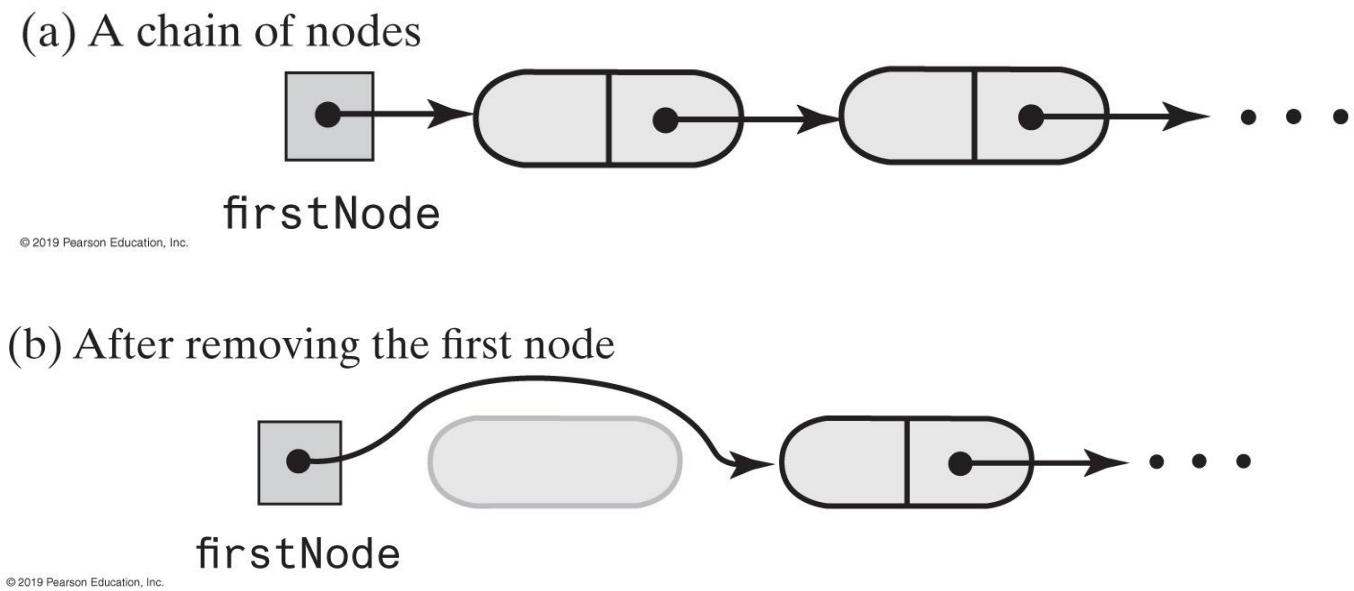
- Possible cases
  - Removing the first node
  - Removing a node other than first one

Removing a Node

- Steps for removing the first node.

Set firstNode *to the link in the first node;* firstNode *now either references the second node or is* null *if the chain had only one node.*

Since all references to the first node no longer exist, the system automatically recycles the first node's memory.



Removing the first node from a chain

- Removing a node other than the first one.

Removing a Node

Let nodeBefore *reference the node before the one to be removed.*

Set nodeToRemove *to* nodeBefore*'s link;* nodeToRemove *now references the node to be removed.*

Set nodeAfter *to* nodeToRemove*'s link;* nodeAfter *now either references the node after the one to be removed or is* null.

Set nodeBefore*'s link to* nodeAfter. *(*nodeToRemove *is now disconnected from the chain.)*

Set nodeToRemove *to* null.

Since all references to the disconnected node no longer exist, the system automatically recycles the node's memory.

Removing a Node

- Operations on a chain depended on the method `getNodeAt`

```
// Returns a reference to the node at a given position.
// Precondition: The chain is not empty;
//               1 <= givenPosition <= numberOfEntries.
private Node getNodeAt(int givenPosition)
{
   // Assertion: (firstNode != null) &&
   //            (1 <= givenPosition) && (givenPosition <= numberOfEntries)
   Node currentNode = firstNode;
   // Traverse the chain to locate the desired node
   // (skipped if givenPosition is 1)
   for (int counter = 1; counter < givenPosition; counter++)
      currentNode = currentNode.getNextNode();
   // Assertion: currentNode != null
   return currentNode;
} // end getNodeAt
```

Data Fields and Constructor (Part 1)

- LISTING 12-1 An outline of the class `LList`

```
/** A linked implemention of the ADT list. */
public class LList<T> implements ListInterface<T>
{
      private Node firstNode;          // Reference to first node of chain
      private int  numberOfEntries;

      public LList()
      {
      initializeDataFields();
      } // end default constructor

      public void clear()
      {
      initializeDataFields();
      } // end clear

/*  < Implementations of the public methods add, remove, replace, getEntry, contains,
     getLength, isEmpty, and toArray go here. >
  . . . */

  // Initializes the class's data fields to indicate an empty list.
  private void initializeDataFields()
  {
      firstNode = null;
      numberOfEntries = 0;
```

Data Fields and Constructor (Part 2)

- Return

```
// Returns a reference to the node at a given position.
// Precondition: The chain is not empty;
//               1 <= givenPosition <= numberOfEntries.
private Node getNodeAt(int givenPosition)
{
   // Assertion: (firstNode != null) &&
   //            (1 <= givenPosition) && (givenPosition <= numberOfEntries)
   Node currentNode = firstNode;

   // Traverse the chain to locate the desired node
   // (skipped if givenPosition is 1)
   for (int counter = 1; counter < givenPosition; counter++)
      currentNode = currentNode.getNextNode();
   // Assertion: currentNode != null
   return currentNode;
} // end getNodeAt

  private class Node
  {
                        // < Implementation of private inner class Node >
```

Adding to the End of the List

- The method `add` assumes method `getNodeAt`

```
public void add(T newEntry)
{
   Node newNode = new Node(newEntry);

   if (isEmpty())
      firstNode = newNode;
   else                          // Add to end of nonempty list
   {
      Node lastNode = getNodeAt(numberOfEntries);
      lastNode.setNextNode(newNode); // Make last node reference new node
   } // end if

   numberOfEntries++;
} // end add
```

Adding at a Given Position

- `add` method.

```
public void add(int givenPosition, T newEntry)
{
   if ((givenPosition >= 1) && (givenPosition <= numberOfEntries + 1))
   {
      Node newNode = new Node(newEntry);
      if (givenPosition == 1)              // Case 1
      {
         newNode.setNextNode(firstNode);
         firstNode = newNode;
      }
      else                                                // Case 2: list is not empty
      {                                 // and givenPosition > 1
         Node nodeBefore = getNodeAt(givenPosition - 1);
         Node nodeAfter = nodeBefore.getNextNode();
         newNode.setNextNode(nodeAfter);
         nodeBefore.setNextNode(newNode);
      } // end if
      numberOfEntries++;
   }
   else
      throw new IndexOutOfBoundsException(
            "Illegal position given to add operation.");
} // end add
```

Method `isEmpty`

```
public boolean isEmpty()
{
   boolean result;

   if (numberOfEntries == 0) // Or getLength() == 0
   {
      // Assertion: firstNode == null
      result = true;
   }
   else
   {
      // Assertion: firstNode != null
      result = false;
   } // end if

   return result;
} // end isEmpty
```

Method `toArray`

```
public T[] toArray()
{
   // The cast is safe because the new array contains null entries
   @SuppressWarnings("unchecked")
   T[] result = (T[])new Object[numberOfEntries];

   int index = 0;
   Node currentNode = firstNode;
   while ((index < numberOfEntries) && (currentNode != null))
   {
      result[index] = currentNode.getData();
      currentNode = currentNode.getNextNode();
      index++;
   } // end while

   return result;
} // end toArray
```

Testing Core Methods

```
public static void main(String[] args)
{
   System.out.println("Create an empty list.");
   ListInterface<String> myList = new LList<String>();
   System.out.println("List should be empty; isEmpty returns " +
                     myList.isEmpty() + ".");

   System.out.println("\nTesting add to end:");
   myList.add("15");
   myList.add("25");
   myList.add("35");
   myList.add("45");
   System.out.println("List should contain 15 25 35 45.");
   displayList(myList);
   System.out.println("List should not be empty; isEmpty() returns " +
                     myList.isEmpty() + ".");
   System.out.println("\nTesting clear():");
   myList.clear();
   System.out.println("List should be empty; isEmpty returns " +
                     myList.isEmpty() + ".");
}// end main
```

Testing Core Methods

- of the ADT list

Program Output

```
Create an empty list.
List should be empty; isEmpty returns true.

Testing add to end:
List should contain 15 25 35 45.
The list contains 4 entries, as follows:
15 25 35 45
List should not be empty; isEmpty() returns false.

Testing clear():
List should be empty; isEmpty returns true.
```

`remove` method returns entry it deletes from list

```
public T remove(int givenPosition)
{
   T result = null;                          // Return value
   if ((givenPosition >= 1) && (givenPosition <= numberOfEntries))
   {
      // Assertion: !isEmpty()
      if (givenPosition == 1)                // Case 1: Remove first entry
      {
         result = firstNode.getData();       // Save entry to be removed
         firstNode = firstNode.getNextNode(); // Remove entry
      }
      else                                   // Case 2: Not first entry
      {
         Node nodeBefore = getNodeAt(givenPosition - 1);
         Node nodeToRemove = nodeBefore.getNextNode();
         result = nodeToRemove.getData();    // Save entry to be removed
         Node nodeAfter = nodeToRemove.getNextNode();
         nodeBefore.setNextNode(nodeAfter);  // Remove entry
      } // end if
      numberOfEntries--;                     // Update count
      return result;                         // Return removed entry
   }
```

Continuing the Implementation

```
public T replace(int givenPosition, T newEntry)
{
   if ((givenPosition >= 1) && (givenPosition <= numberOfEntries))
   {
      // Assertion: !isEmpty()
      Node desiredNode = getNodeAt(givenPosition);
      T originalEntry = desiredNode.getData();
      desiredNode.setData(newEntry);
      return originalEntry;
   }
   else
      throw new IndexOutOfBoundsException(
               "Illegal position given to replace operation.");
} // end replace
```

- Replacing a list entry requires us to replace the data portion of a node with other data.

Continuing the Implementation

- Retrieving a list entry is straightforward.

```
public T getEntry(int givenPosition)
{
   if ((givenPosition >= 1) && (givenPosition <= numberOfEntries))
   {
      // Assertion: !isEmpty()
      return getNodeAt(givenPosition).getData();
   }
   else
      throw new IndexOutOfBoundsException(
               "Illegal position given to getEntry operation.");
} // end getEntry
```

Continuing the Implementation

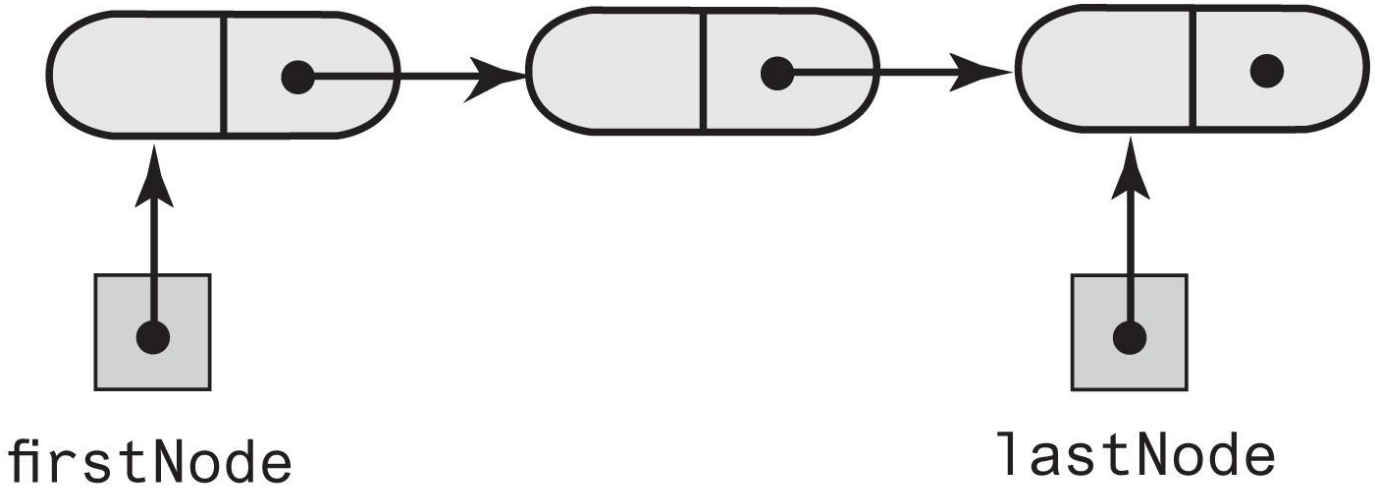
```
public boolean contains(T anEntry)
{
   boolean found = false;
   Node currentNode = firstNode;

   while (!found && (currentNode != null))
   {
      if (anEntry.equals(currentNode.getData()))
         found = true;
      else
         currentNode = currentNode.getNextNode();
   } // end while

   return found;
} // end contains
```

- Checking to see if an entry is in the list, the method `contains`.

A Refined Linked Implementation

- A linked chain with both a head reference and a tail reference

```
private Node firstNode;       // Head reference to first node
private Node lastNode;        // Tail reference to last node
private int  numberOfEntries; // Number of entries in list
```



© 2019 Pearson Education, Inc.

A Refined Linked Implementation

- Revision of the first `add` method

```
public void add(T newEntry)
{
   Node newNode = new Node(newEntry);

   if (isEmpty())
      firstNode = newNode;
   else
      lastNode.setNextNode(newNode);

   lastNode = newNode;
   numberOfEntries++;
}  // end add
```

A Refined Linked Implementation - refined add by position

```
public void add(int givenPosition, T newEntry) {
  if ((givenPosition >= 1) && (givenPosition <= numberOfEntries + 1))
  {
    Node newNode = new Node(newEntry);
    if (isEmpty())
    {
      firstNode = newNode;
      lastNode = newNode;
    }
    else if (givenPosition == 1)
    {
      newNode.setNextNode(firstNode);
      firstNode = newNode;
    }
    else if (givenPosition == numberOfEntries + 1)
    {
      lastNode.setNextNode(newNode);
      lastNode = newNode;
    }
    else {
      Node nodeBefore = getNodeAt(givenPosition - 1);
      Node nodeAfter = nodeBefore.getNextNode();
      newNode.setNextNode(nodeAfter);
      nodeBefore.setNextNode(newNode);
    } // end if
    numberOfEntries++;
  }
  else
    throw new IndexOutOfBoundsException(
            "Illegal position given to add operation.");
} // end add
```

A Refined Linked Implementation — refined `remove`

```
public T remove(int givenPosition) {
   T result = null;                        // Return value
   if ((givenPosition >= 1) && (givenPosition <= numberOfEntries))
   {
   // Assertion: !isEmpty()
      if (givenPosition == 1)              // Case 1: Remove first entry
      {
         result = firstNode.getData();       // Save entry to be removed
         firstNode = firstNode.getNextNode();
         if (numberOfEntries == 1)
            lastNode = null;               // Solitary entry was removed
      }
      else                                 // Case 2: Not first entry
      {
         Node nodeBefore = getNodeAt(givenPosition - 1);
         Node nodeToRemove = nodeBefore.getNextNode();
         Node nodeAfter = nodeToRemove.getNextNode();
         nodeBefore.setNextNode(nodeAfter);
         result = nodeToRemove.getData();
         if (givenPosition == numberOfEntries)
            lastNode = nodeBefore;         // Last node was removed
      } // end if
      numberOfEntries--;
   }
   else
      throw new IndexOutOfBoundsException(
            "Illegal position given to remove operation.");

   return result;                          // Return removed entry
} // end remove
```

Efficiency of Using a Chain

`add(newEntry)`	O(1)	O(*n*)	O(1)
`add(givenPosition, newEntry)`	O(*n*); O(n); O(1)	O(1); O(*n*)	O(1); O(*n*); O(1)
`toArray()`	O(*n*)	O(*n*)	O(*n*)
`remove(givenPosition)`	O(*n*); O(*n*); O(1)	O(1); O(*n*)	O(1); O(*n*)
`replace(givenPosition, newEntry)`	O(1)	O(1); O(*n*)	O(1); O(*n*); O(1)
`getEntry(givenPosition)`	O(1)	O(1); O(*n*)	O(1); O(*n*); O(1)
`contains(anEntry)`	O(*n*)	O(*n*)	O(*n*)
`clear(), getLength(), isEmpty()`	O(1)	O(1)	O(1)

Java Class Library: The Class `LinkedList`

- Implements the interface **`List`**
- **`LinkedList`** defines more methods than are in the interface `List`
- You can use the class **`LinkedList`** as implementation of ADT
  - **queue**
  - **deque**
  - or **list**.